Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Ростовский государственный экономический университет (РИНХ)»

ОТЧЁТ

по лабораторной работе № 1.4

«Создание компонентов MVC»

по дисциплине «Разработка и сопровождение программных систем»

Вариант 13

Выполнил: студент группы ИСТ-321
Хасаев Умахан

Ростов-на-Дону
2026

Репозиторий с исходным кодом: https://github.com/warhogz/software-development-and-maintenance

Цель работы

Изучить реализацию шаблона MVC в JavaFX-приложении: создать классы-модели, использовать их в коллекции ObservableList и отобразить данные в компоненте TableView с помощью контроллера.

Задание

1. Создать класс-модель.

2. Использовать класс-модель в коллекции ObservableList.

3. Отобразить данные в компоненте TableView с помощью контроллеров.

Индивидуальное задание (вариант 13): предметная область «Области – Районы».

Таблица 1 «Области»: название, площадь, административный центр, глава.

Таблица 2 «Районы»: название, площадь, административный центр, глава.

Ход работы

Работа является продолжением приложения RegionApp, созданного в лабораторной работе 1.3 (структура проекта и пользовательский интерфейс, спроектированный в Scene Builder). Проект собирается с помощью Maven, используется JavaFX 21. Структура проекта приведена в листинге 1.

Листинг 1 – Структура проекта

```
lab1_4
├── pom.xml
└── src/main
    ├── java
    │   ├── module-info.java
    │   └── ru/lab/regionapp
    │       ├── MainApp.java
    │       ├── model
    │       │   ├── District.java
    │       │   └── Region.java
```

```
│       └── view
│           └── DistrictOverviewController.java
└── resources/ru/lab/regionapp/view
    ├── DistrictOverview.fxml
    └── RootLayout.fxml
```

1. Создание классов-моделей. В пакете model созданы два класса-модели: Region (область) и District (район). Поля классов объявлены как свойства JavaFX (StringProperty, DoubleProperty, ObjectProperty), поэтому элементы интерфейса автоматически получают уведомления об изменении данных. Для каждого свойства есть методы get, set и метод, возвращающий само свойство. Район хранит ссылку на область, в которую он входит. Код классов приведён в листингах 2 и 3.

Листинг 2 – Класс-модель Region

```
package ru.lab.regionapp.model;

import javafx.beans.property.DoubleProperty;
import javafx.beans.property.SimpleDoubleProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.beans.property.StringProperty;

/**
 * Класс-модель для области (Region).
 */
public class Region {

    private final StringProperty name;        // Название
    private final DoubleProperty area;        // Площадь, км²
    private final StringProperty adminCenter; // Административный центр
    private final StringProperty head;        // Глава

    /**
     * Конструктор по умолчанию.
     */
    public Region() {
        this(null, 0, null, null);
    }

    public Region(String name, double area, String adminCenter, String head) {
        this.name = new SimpleStringProperty(name);
        this.area = new SimpleDoubleProperty(area);
        this.adminCenter = new SimpleStringProperty(adminCenter);
        this.head = new SimpleStringProperty(head);
    }

    public String getName() {
        return name.get();
    }

    public void setName(String name) {
        this.name.set(name);
    }
```

```
    public StringProperty nameProperty() {
        return name;
    }

    public double getArea() {
        return area.get();
    }

    public void setArea(double area) {
        this.area.set(area);
    }

    public DoubleProperty areaProperty() {
        return area;
    }

    public String getAdminCenter() {
        return adminCenter.get();
    }

    public void setAdminCenter(String adminCenter) {
        this.adminCenter.set(adminCenter);
    }

    public StringProperty adminCenterProperty() {
        return adminCenter;
    }

    public String getHead() {
        return head.get();
    }

    public void setHead(String head) {
        this.head.set(head);
    }

    public StringProperty headProperty() {
        return head;
    }

    @Override
    public String toString() {
        return getName();
    }
}
```

Листинг 3 – Класс-модель District

```
package ru.lab.regionapp.model;

import javafx.beans.property.DoubleProperty;
import javafx.beans.property.ObjectProperty;
import javafx.beans.property.SimpleDoubleProperty;
import javafx.beans.property.SimpleObjectProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.beans.property.StringProperty;

/**
 * Класс-модель для района (District).
 */
public class District {
```

```
    private final StringProperty name;           // Название
    private final DoubleProperty area;           // Площадь, км²
    private final StringProperty adminCenter;    // Административный центр
    private final StringProperty head;           // Глава
    private final ObjectProperty<Region> region; // Область, в которую входит район


    /**
     * Конструктор по умолчанию.
     */
    public District() {
        this(null, 0, null, null, null);
    }


    public District(String name, double area, String adminCenter, String head, Region
region) {
        this.name = new SimpleStringProperty(name);
        this.area = new SimpleDoubleProperty(area);
        this.adminCenter = new SimpleStringProperty(adminCenter);
        this.head = new SimpleStringProperty(head);
        this.region = new SimpleObjectProperty<>(region);
    }


    public String getName() {
        return name.get();
    }


    public void setName(String name) {
        this.name.set(name);
    }


    public StringProperty nameProperty() {
        return name;
    }


    public double getArea() {
        return area.get();
    }


    public void setArea(double area) {
        this.area.set(area);
    }


    public DoubleProperty areaProperty() {
        return area;
    }


    public String getAdminCenter() {
        return adminCenter.get();
    }


    public void setAdminCenter(String adminCenter) {
        this.adminCenter.set(adminCenter);
    }


    public StringProperty adminCenterProperty() {
        return adminCenter;
    }


    public String getHead() {
        return head.get();
    }


    public void setHead(String head) {
```

```
        this.head.set(head);
    }

    public StringProperty headProperty() {
        return head;
    }

    public Region getRegion() {
        return region.get();
    }

    public void setRegion(Region region) {
        this.region.set(region);
    }

    public ObjectProperty<Region> regionProperty() {
        return region;
    }
}
```

2. Использование ObservableList. В главном классе MainApp объявлены наблюдаемые коллекции ObservableList для областей и районов, которые заполняются тестовыми данными в конструкторе. После загрузки макета DistrictOverview.fxml главный класс передаёт контроллеру ссылку на себя (листинг 4).

Листинг 4 – Класс MainApp

```
package ru.lab.regionapp;

import java.io.IOException;

import javafx.application.Application;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.scene.layout.AnchorPane;
import javafx.scene.layout.BorderPane;
import javafx.stage.Stage;
import ru.lab.regionapp.model.District;
import ru.lab.regionapp.model.Region;
import ru.lab.regionapp.view.DistrictOverviewController;

/**
 * Лабораторная работа 1.4. Компоненты MVC (вариант 13: Области – Районы).
 */
public class MainApp extends Application {

    private Stage primaryStage;
    private BorderPane rootLayout;

    /**
     * Данные в виде наблюдаемых списков областей и районов.
     */
    private final ObservableList<Region> regionData =
FXCollections.observableArrayList();
```

```
    private final ObservableList<District> districtData =
FXCollections.observableArrayList();

    /**
     * Конструктор: заполняем списки тестовыми данными.
     */
    public MainApp() {
        Region nn = new Region("Нижегородская область", 76624, "Нижний Новгород",
"Петров П.П.");
        Region vl = new Region("Владимирская область", 29084, "Владимир", "Соколов
В.М.");
        Region rs = new Region("Ростовская область", 100967, "Ростов-на-Дону",
"Захаров О.Л.");
        regionData.addAll(nn, vl, rs);

        districtData.add(new District("Арзамасский район", 1923, "Арзамас", "Иванов
И.И.", nn));
        districtData.add(new District("Богородский район", 1300, "Богородск", "Смирнов
А.В.", nn));
        districtData.add(new District("Кстовский район", 1203, "Кстово", "Кузнецов
Д.С.", nn));
        districtData.add(new District("Павловский район", 1077, "Павлово", "Орлова
Е.Н.", nn));
        districtData.add(new District("Ковровский район", 1999, "Ковров", "Морозов
К.А.", vl));
        districtData.add(new District("Суздальский район", 1263, "Суздаль", "Волкова
И.Г.", vl));
        districtData.add(new District("Муромский район", 1450, "Муром", "Лебедев
Р.О.", vl));
        districtData.add(new District("Аксайский район", 1210, "Аксай", "Белов С.Н.",
rs));
        districtData.add(new District("Азовский район", 2864, "Азов", "Ершова Т.В.",
rs));
    }

    /**
     * Возвращает данные в виде наблюдаемого списка областей.
     */
    public ObservableList<Region> getRegionData() {
        return regionData;
    }

    /**
     * Возвращает данные в виде наблюдаемого списка районов.
     */
    public ObservableList<District> getDistrictData() {
        return districtData;
    }

    @Override
    public void start(Stage primaryStage) {
        this.primaryStage = primaryStage;
        this.primaryStage.setTitle("Области и районы");

        initRootLayout();

        showDistrictOverview();
    }

    /**
     * Инициализирует корневой макет.
     */
    public void initRootLayout() {
        try {
            // Загружаем корневой макет из fxml файла.
            FXMLLoader loader = new FXMLLoader();
```

```
            loader.setLocation(MainApp.class.getResource("view/RootLayout.fxml"));
            rootLayout = (BorderPane) loader.load();

            // Отображаем сцену, содержащую корневой макет.
            Scene scene = new Scene(rootLayout);
            primaryStage.setScene(scene);
            primaryStage.show();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    /**
     * Показывает в корневом макете сведения о районах.
     */
    public void showDistrictOverview() {
        try {
            // Загружаем сведения о районах.
            FXMLLoader loader = new FXMLLoader();
loader.setLocation(MainApp.class.getResource("view/DistrictOverview.fxml"));
            AnchorPane districtOverview = (AnchorPane) loader.load();

            // Помещаем сведения о районах в центр корневого макета.
            rootLayout.setCenter(districtOverview);

            // Даём контроллеру доступ к главному приложению.
            DistrictOverviewController controller = loader.getController();
            controller.setMainApp(this);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    /**
     * Возвращает главную сцену.
     */
    public Stage getPrimaryStage() {
        return primaryStage;
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

3. Отображение данных в TableView. Контроллер DistrictOverviewController связан с FXML-разметкой через атрибут fx:controller, а элементы интерфейса – через атрибуты fx:id и аннотацию @FXML (листинг 5). В методе initialize() для столбцов таблицы задаются фабрики значений ячеек с помощью лямбда-выражений, а в методе setMainApp() таблица связывается с коллекцией районов. При выборе строки таблицы подробные сведения о районе выводятся в правой части окна. Код контроллера приведён в листинге 6.

Листинг 5 – Фрагмент файла DistrictOverview.fxml

```
<AnchorPane ... fx:controller="ru.lab.regionapp.view.DistrictOverviewController">
  ...
  <TableView fx:id="districtTable" ...>
    <columns>
      <TableColumn fx:id="nameColumn" prefWidth="140.0" text="Район" />
      <TableColumn fx:id="regionColumn" prefWidth="140.0" text="Область" />
    </columns>
  </TableView>
  ...
  <Label fx:id="nameLabel" text="Label" GridPane.columnIndex="1" />
  <Label fx:id="areaLabel" text="Label" GridPane.columnIndex="1" GridPane.rowIndex="1"
/>
  <Label fx:id="adminCenterLabel" text="Label" GridPane.columnIndex="1"
GridPane.rowIndex="2" />
  <Label fx:id="headLabel" text="Label" GridPane.columnIndex="1" GridPane.rowIndex="3"
/>
  <Label fx:id="regionLabel" text="Label" GridPane.columnIndex="1"
GridPane.rowIndex="4" />
```

Листинг 6 – Класс DistrictOverviewController

```
package ru.lab.regionapp.view;


import javafx.fxml.FXML;
import javafx.scene.control.Label;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import ru.lab.regionapp.MainApp;
import ru.lab.regionapp.model.District;


/**
 * Контроллер для DistrictOverview.fxml
 */
public class DistrictOverviewController {


    @FXML
    private TableView<District> districtTable;
    @FXML
    private TableColumn<District, String> nameColumn;
    @FXML
    private TableColumn<District, String> regionColumn;


    @FXML
    private Label nameLabel;
    @FXML
    private Label areaLabel;
    @FXML
    private Label adminCenterLabel;
    @FXML
    private Label headLabel;
    @FXML
    private Label regionLabel;


    // Ссылка на главное приложение.
    private MainApp mainApp;


    /**
     * Конструктор вызывается раньше метода initialize().
     */
    public DistrictOverviewController() {
    }
```

```
    /**
     * Инициализация класса-контроллера. Этот метод вызывается автоматически
     * после того, как fxml-файл будет загружен.
     */
    @FXML
    private void initialize() {
        // Инициализация таблицы районов с двумя столбцами.
        nameColumn.setCellValueFactory(cellData ->
cellData.getValue().nameProperty());
        regionColumn.setCellValueFactory(cellData ->
cellData.getValue().getRegion().nameProperty());


        // Очистка подробной информации о районе.
        showDistrictDetails(null);


        // Слушаем изменения выбора и при изменении отображаем подробную информацию о
районе.
        districtTable.getSelectionModel().selectedItemProperty().addListener(
                (observable, oldValue, newValue) -> showDistrictDetails(newValue));
    }


    /**
     * Вызывается главным приложением, которое даёт на себя ссылку.
     */
    public void setMainApp(MainApp mainApp) {
        this.mainApp = mainApp;


        // Добавление в таблицу данных из наблюдаемого списка.
        districtTable.setItems(mainApp.getDistrictData());
    }


    /**
     * Заполняет текстовые поля, отображая подробности о районе.
     * Если указанный район = null, то все текстовые поля очищаются.
     */
    private void showDistrictDetails(District district) {
        if (district != null) {
            nameLabel.setText(district.getName());
            areaLabel.setText(String.format("%.0f км²", district.getArea()));
            adminCenterLabel.setText(district.getAdminCenter());
            headLabel.setText(district.getHead());
            regionLabel.setText(district.getRegion().getName());
        } else {
            nameLabel.setText("");
            areaLabel.setText("");
            adminCenterLabel.setText("");
            headLabel.setText("");
            regionLabel.setText("");
        }
    }
}
```

Результаты работы программы

Результат работы приложения показан на рисунках 1 и 2.

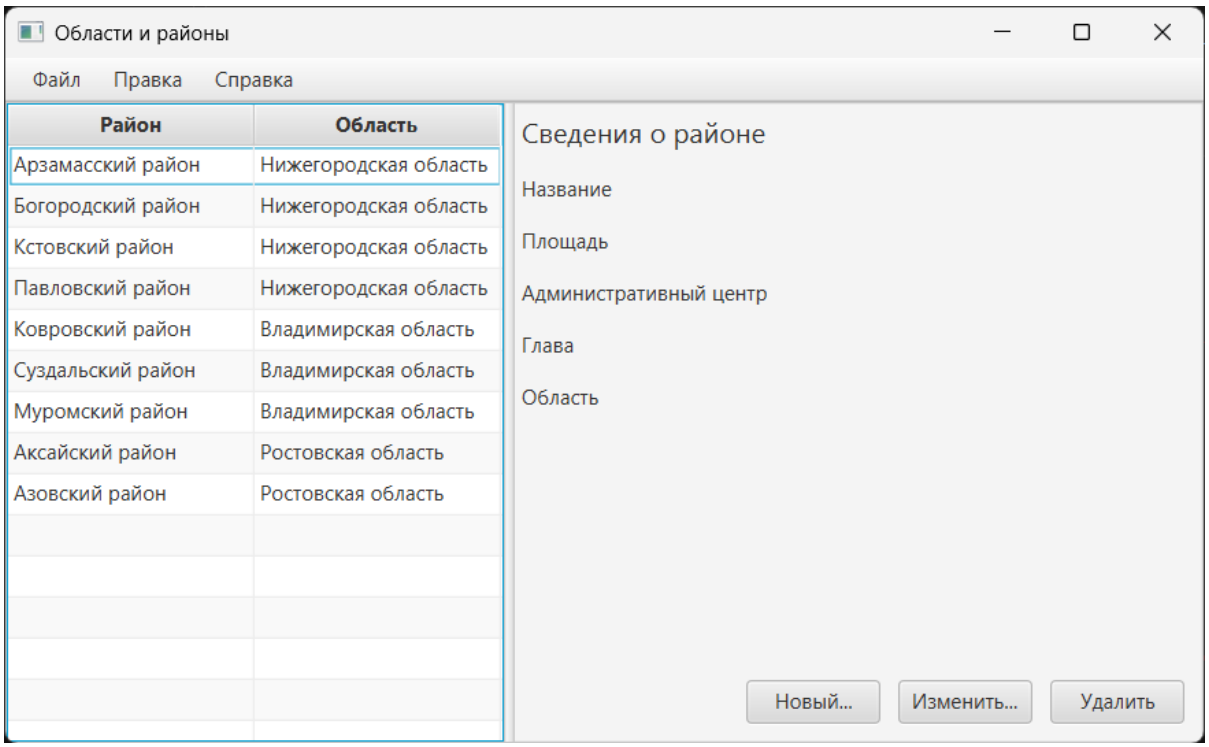


Рисунок 1 – Главное окно приложения со списком районов

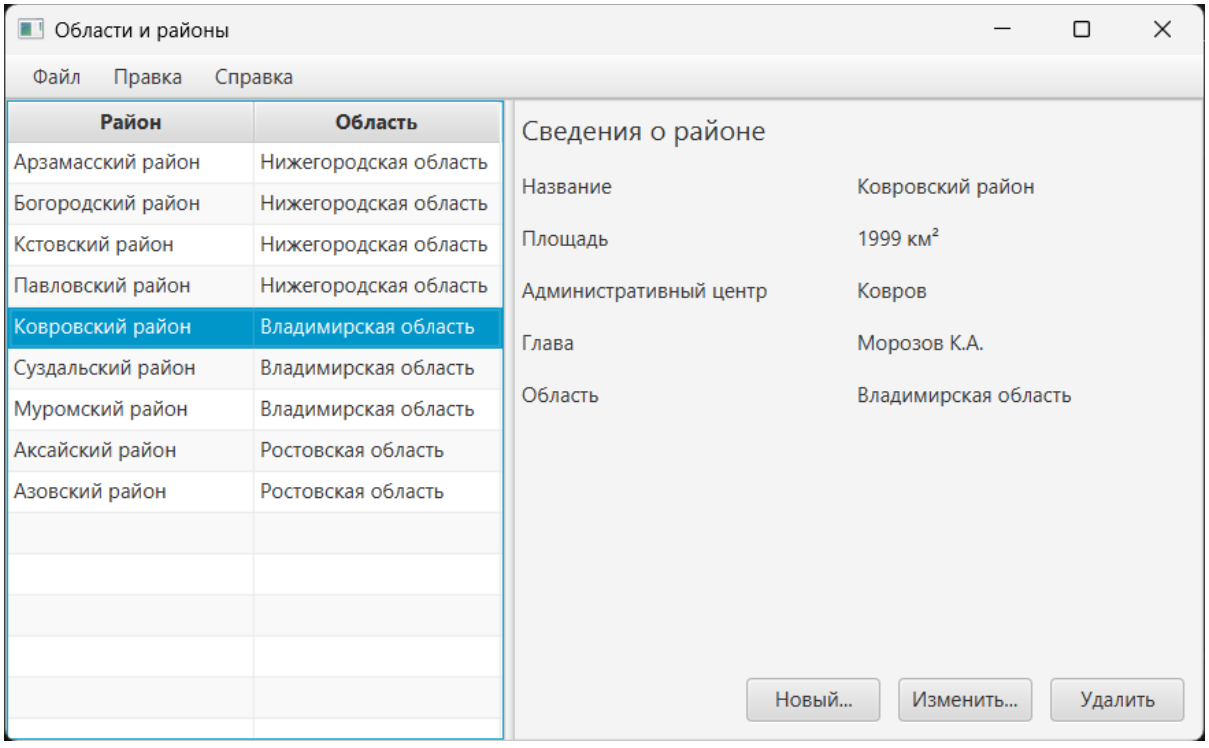


Рисунок 2 – Отображение сведений о выбранном районе

Вывод

В ходе выполнения лабораторной работы были созданы классы-модели Region и District со свойствами JavaFX, данные о районах размещены в наблюдаемой коллекции ObservableList и отображены в компоненте

TableView с помощью контроллера DistrictOverviewController. Приложение построено по шаблону MVC: модель (пакет model), представление (FXML-файлы) и контроллер разделены, что упрощает дальнейшее развитие программы.